SoundSight

AI-Powered Sound Awareness for the Hearing Impaired

What is it?

SoundSight is a combined AI + IoT system that helps hearing-impaired people perceive environmental sounds. A low-cost ESP32 microcontroller with a digital microphone continuously captures audio and sends it to a Python/FastAPI backend. A trained ML classification model identifies the sound (fire alarm, doorbell, baby crying, etc.), and the device responds instantly with color-coded LEDs and vibration alerts. A locally-hosted LLM — orchestrated by LangChain, grounded by RAG, and backed by memory of the user's baselines — adds contextual intelligence: generating daily reports, explaining anomalies, and parsing alert rules written in natural language.

Why does it matter?

430 million people worldwide have disabling hearing loss. Existing assistive sound devices are expensive, single-purpose, and lack intelligence. SoundSight provides an affordable (~$15/node), multi-sound, context-aware, privacy-preserving alternative — with the LLM running locally, no user audio leaves the machine.

Technical Overview

IoT Layer	AI Layer	Application Layer
• ESP32 + INMP441 mic • WS2812B LEDs • Vibration motor • Firmware in C/C++ • WiFi (HTTP POST)	• ML: audio classification (MFCC / mel-spectrogram) • LLM: Ollama (local) • LangChain orchestration • MCP service layer • RAG via ChromaDB • Memory: user baselines + session buffer • Prompt testing with Promptfoo	• Python 3.11 + FastAPI • React + TypeScript frontend • WebSocket (real-time events) • PostgreSQL / SQLite • pytest + Vitest unit tests

AI Apps Exam Coverage

- **ML fundamentals:** audio feature engineering (MFCCs / mel-spectrograms), classification model training and evaluation, Python-native inference.
- **LLM application (non-chatbot):** event interpretation, automated report generation, anomaly narration, natural-language → structured rule parsing, onboarding profiler.
- **Ollama-hosted LLM** controlled via system prompting, few-shot examples, chain-of-thought, and structured (Pydantic) output.
- **LangChain** orchestrates all five LLM roles as chains and agents, targeting the local Ollama model.
- **MCP service layer** exposes backend tools (query_events, get_device_status, get_user_baseline, …) to LangChain agents.
- **RAG** over event history, home knowledge, and a sound-class knowledge base (ChromaDB + sentence-transformers).
- **Memory** layer: short-term session buffer for interactive flows + long-term vector/summary memory for user baselines.
- **Promptfoo** regression suite for the rule parser and other structured-output chains.
- **Unit tests** with pytest (audio preprocess, classifier, endpoints, chains, MCP tools, RAG retrievers) and Vitest + RTL (React components).

IoT Exam Coverage

- Real-time sensor data capture (I2S audio from INMP441), firmware in C/C++.
- WiFi communication with the FastAPI backend (HTTP POST).
- Actuator control — color-coded WS2812B LEDs and vibration motor per event severity.
- Multi-node architecture: each ESP32 self-registers and is assigned to a room.
- Edge device configuration via an AP-mode config portal on first boot.

Team Members

Stefan Rubelov • Samuel Stiksa